T R A V E L L E R S I N

Architecture Gap Analysis

& Improved System Design

Communication • Payment • Agency Operations • Edge Cases • Implementation

Based on repo analysis: github.com/Aryan-one6/tripSync_UI

April 2026 • v3.0

Honest Gap Audit: What's Wrong Right Now

After re-analyzing the repo (422-line Prisma schema, 595-line payment service, 771-line chat service, 461-line offer service, and the full frontend), here are the specific gaps that will cause real business problems:

Gap	Current State	Business Impact	Severity
Agency is single-person	Agency model has only ownerId (one user). No team members, no roles.	Real agencies have 3–10 people. Owner must do everything alone. They'll use WhatsApp internally and leak conversations.	Critical
Payment has no convenience fee split	Payment model stores amount (trip cost only). No line item for ₹299 fee. No Razorpay Route/Transfer integration.	Platform captures zero direct revenue. The 8% commission deduction has no implementation path.	Critical
No phone/contact masking	Chat service has no content filtering. Phone numbers, UPI IDs, email addresses pass through freely.	Users and agencies exchange contacts in first 5 messages. All future communication moves to WhatsApp. Platform loses control.	Critical
Escrow has no partial/advance release	Payment model has tranche1/tranche2 booleans but the release logic doesn't integrate with Razorpay Route.	Agencies with cash-flow pressure can't operate. They demand advance payment outside the platform.	High
No invoice/receipt system	No Invoice or Receipt model. No downloadable payment proof.	Agencies can't reconcile with their accounting. Users have no proof of purchase. Both sides lose trust.	High
No cancellation policy enforcement	Offer model has cancellationPolicy as free text. No structured enforcement.	When disputes happen, there's no system to calculate refund amounts. Manual intervention required every time.	High
No agency wallet/ledger	No AgencyWallet or AgencyTransaction model. Agency can't see pending/released/earned amounts.	Agency has no financial visibility. They call your support team daily asking 'Where's my money?'	High
Package flow has no payment path	getGroupPaymentContext() only works for plans with selectedOffer. Packages (agency-created) don't trigger payment.	Users can browse and 'join' agency packages but can never pay. The entire package flow is non-transactional.	Critical
No webhook reconciliation	razorpayWebhookHandler exists but no reconciliation job. If webhook is missed, payment status is stuck.	User paid but system shows PENDING. User panics. Agency never gets confirmed booking. Trust destroyed.	High
Chat unlocks too early	Direct messages unlock with just 'trip participation or agency with live offers'. No payment gate.	Agency and user can chat freely before any money is on the platform. They negotiate offline and bypass entirely.	Medium

1. Communication System: Keeping Everyone On-Platform

1.1 The Fundamental Problem

Your chat service (771 lines) already handles group chat, DMs, polls, and system messages. Technically it works. But it has zero anti-disintermediation controls. The moment an agency types their phone number, the platform loses the customer forever.

1.2 Contact Masking System

Add a content filter middleware that runs on every outgoing chat message (both group and DM). The filter detects and masks contact information before it reaches other users.

What to Detect and Mask

Pattern	Example Input	Masked Output	Detection Method
Phone numbers	Call me at 9876543210	Call me at 98****3210 [Contact shared via platform]	Regex: 10-digit Indian mobile starting with 6-9, with optional +91/0 prefix
UPI IDs	Pay at rahul@paytm	Pay at r****@paytm [Payment request detected]	Regex: word@word pattern
Email addresses	Email me at rahul@gmail.com	Email me at r****@gmail.com [Contact shared via platform]	Regex: standard email pattern
Bank details	IFSC: SBIN0001234	IFSC: SBIN****234 [Bank details detected]	Regex: 11-char IFSC pattern
WhatsApp references	Text me on WhatsApp	Text me on WhatsApp [Use platform chat for safety]	Keyword: whatsapp, telegram, signal

Implementation Approach

Add a sanitizeChatContent() function in chat/service.ts that runs BEFORE saving any message. The original content is stored in a metadata.originalContent field (for admin review) but the masked version is what users see.

```
// In chat/service.ts - add before createGroupMessage()

const PHONE_REGEX = /(?:\+?91[\s-]?)?[6-9]\d{9}/g;
const UPI_REGEX = /[a-zA-Z0-9._-]+@[a-zA-Z]{2,}/g;
const EMAIL_REGEX = /[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}/g;
const IFSC_REGEX = /[A-Z]{4}0[A-Z0-9]{6}/g;
const BYPASS_KEYWORDS = /whatsapp|telegram|signal|gpay|phonepe|paytm/gi;

interface SanitizeResult {
  sanitized: string;
  flagged: boolean;
  originalContent?: string;
}

function sanitizeChatContent(content: string): SanitizeResult {
  let flagged = false;
  let sanitized = content;

  // Mask phone numbers
  sanitized = sanitized.replace(PHONE_REGEX, (match) => {
    flagged = true;
```

```
    return match.slice(0,2) + '****' + match.slice(-4);
  });

  // Mask UPI/email (skip if already masked)
  sanitized = sanitized.replace(UPI_REGEX, (match) => {
    if (match.includes('****')) return match;
    flagged = true;
    const [local, domain] = match.split('@');
    return local[0] + '****@' + domain;
  });

  // Flag bypass keywords
  if (BYPASS_KEYWORDS.test(sanitized)) flagged = true;

  return { sanitized, flagged, originalContent: flagged ? content : undefined };
}
```

When a message is flagged, inject a system message immediately after it:

- '⚠️ For your safety, all payments and coordination should happen through TravellersIn. External transactions are not covered by our escrow protection or refund policy.'

1.3 Chat Access Rules (When Chat Unlocks)

Currently, your assertDirectConversationEligibility() checks for trip participation OR agency ownership. This is too loose. Tighten it:

Context	When Chat Unlocks	What's Allowed	What's Blocked
User–User (group chat)	User joins a plan/package (INTERESTED status)	Text, polls, system messages	File sharing (until CONFIRMED), contact info always masked
User–Agency (DM)	Agency submits offer on user's plan, OR user joins agency's package	Text, offer cards, system messages	Contact info masked. No file sharing until payment completed.
Agency–Group (after acceptance)	Offer accepted (CONFIRMING state)	Text, document sharing (itinerary, bookings), system messages	Contact info masked until trip is CONFIRMED and all paid
Post-payment (CONFIRMED)	All members paid	Full chat: text, files, documents, contact info partially unmasked (agency rep phone only, via platform)	Direct bank/UPI details still masked
Post-trip (COMPLETED)	Trip finished	Read + write for 30 days. Reviews. Photo sharing.	After 30 days: read-only

1.4 Agency Contact Sharing (The Safe Way)

After CONFIRMED status (everyone paid), the platform shares the agency's contact details through a special system card in the chat. This is controlled — the platform decides when and how contact info is revealed:

- A 'Trip Contact Card' system message appears showing: Agency representative name, a platform-masked phone number (calls route through Exotel/Knowlarity virtual number), meeting point details, and emergency contact.
- The virtual number setup means: users can call the agency, but the call is logged and routed through the platform. If a dispute happens, you have call records.

- This is how Airbnb, Uber, and Urban Company handle contact masking — it's a proven pattern.

1.5 Incentives to Stay On-Platform

Why Users Stay

- Escrow protection: 'Your ₹12,500 is protected. If the agency cancels, you get a full refund. This protection only works for platform payments.'
- Trip documents: itinerary, hotel bookings, permits are pinned in the chat. Leaving the platform means losing access to these.
- Reviews: only platform-transacted trips can be reviewed. Reviews are the #1 trust signal for future trips.
- Insurance eligibility: (Phase 3) platform-booked trips are eligible for travel insurance. Off-platform bookings are not.
- Dispute resolution: if something goes wrong, the platform mediates. Off-platform = on your own.

Why Agencies Stay

- Qualified leads: structured plans with dates, budget, headcount. Not vague WhatsApp inquiries.
- Lower commission than OTAs: 8% vs MakeMyTrip's 15–25%. This IS the pitch.
- Payment guarantee: escrow means the agency gets paid even if users are flaky.
- Reviews and ratings: agencies build public social proof that attracts more bookings.
- Analytics: demand heatmaps, bid win rates, conversion data. No other platform provides this to small agencies.
- Agency wallet: clear visibility of pending, released, and earned amounts. No guessing.

2. Payment Architecture: The Complete Redesign

2.1 What's Broken in the Current Implementation

Your payment service (595 lines) handles order creation, Razorpay checkout, webhook verification, mock capture, and escrow release. This is solid infrastructure. But it's missing the business logic layer on top:

- No convenience fee (₹299) as a separate line item — the amount field just stores the trip cost.
- No Razorpay Route / Transfer integration — the escrow release function updates database booleans but doesn't actually move money via Razorpay's API.
- No split between platform revenue and agency payout — all money goes to one Razorpay merchant account.
- The package flow (agency-created trips) doesn't connect to payment at all — getGroupPaymentContext() requires a selectedOffer which packages don't have.

2.2 Hybrid Payment Model

Mode 1: Trust Mode (Default for All Agencies)

Full escrow. Agency money is held by Razorpay until trip milestones are confirmed. This is the safe default.

Aspect	Detail
Who qualifies	All agencies by default. New agencies always start here.
How it works	100% of trip cost goes to Razorpay escrow. Released in 2 tranches: 50% on trip start, 50% on trip end + 48h.
Agency experience	Agency sees 'Payout scheduled' in their wallet. Clear dates for when money arrives.
Platform commission	8% deducted from each tranche before release. Agency receives 92% of trip cost.
Risk to platform	Low. Money is in Razorpay's escrow, not your bank account.
Risk to agency	Cash flow pressure. Agency must front hotel/transport costs before receiving payout.

Mode 2: Pro Mode (Earned by Verified Agencies)

Faster payouts with advance release. Reduces agency cash-flow pressure.

Aspect	Detail
Who qualifies	VERIFIED agencies with: 10+ completed trips, 4.0+ rating, 6+ months active, zero disputes in last 90 days.
How it works	30% of trip cost released on booking confirmation (all members paid). 30% on trip start. 32% on trip end + 48h. 8% commission deducted across tranches.
Agency experience	Agency gets early cash flow for logistics. Feels like a trusted partnership, not a hostage situation.
Platform commission	Same 8%. Deducted proportionally from each tranche.
Risk to platform	Higher. If agency disappears after getting 30% advance, platform must refund users from its own pocket. Mitigated by trust score + deposit requirement.
Risk to agency	Lower. Better cash flow enables better trips.
Safety net	Pro agencies deposit ₹25,000 refundable security deposit (held by platform). If agency defaults, deposit covers partial refund.

When to Graduate Agency from Trust to Pro

- Automated: system checks eligibility monthly. When criteria met, agency gets a notification: 'Congratulations! You qualify for Pro payouts.'

- Manual override: platform admin can promote or demote agencies. Used for onboarding high-value agencies at launch.
- Demotion: if rating drops below 4.0 or a verified dispute occurs, agency reverts to Trust Mode with 30-day notice.

2.3 Case 1: User Joins Agency Package (No Negotiation)

This flow is currently broken in your codebase. getGroupPaymentContext() requires selectedOffer which packages don't have. Here's the fix:

```
FLOW: User Joins Agency Package

[1] User browses discover feed → taps an agency package
[2] Sees package detail: price, itinerary, inclusions, reviews
[3] Taps 'Join This Trip' → added to group (INTERESTED)
[4] Chat with other members is available (text only, masked)

[5] When group reaches minimum size:
    → Package moves to CONFIRMING
    → Payment prompt appears in group chat for all members
    → 48h countdown starts

[6] User taps 'Pay Now':
    WHAT USER PAYS:
      Package price:              ₹12,500
      + Platform fee:                ₹299
      + GST on fee (18%):             ₹54
      = TOTAL:                    ₹12,853

    WHERE MONEY GOES (Razorpay Route split):
      ₹353 (₹299+GST) → TravellersIn settlement account
      ₹11,500 (92%)  → Agency linked account (escrow)
      ₹1,000 (8%)    → TravellersIn commission account

[7] All members pay → CONFIRMED
    → Group locked. Agency joins chat. Documents shared.

[8] Trip happens → COMPLETED
    → Tranche releases per Trust/Pro mode schedule
```

Database Fix for Package Payments

The getGroupPaymentContext() function needs to handle packages directly. Add a new code path:

```
// In payments/service.ts - modify getGroupPaymentContext()

// CURRENT: only handles plans with selectedOffer
// FIX: add package path

if (membership.group.package) {
  // Package flow: price comes from the package directly
  const pkg = membership.group.package;
  const amount = pkg.basePrice * 100; // paise
  const existingPayment = membership.group.payments
    .find(p => p.userId === userId) ?? null;

  return {
    membership,
```

```
    group: membership.group,
    plan: null,
    package: pkg,
    offer: null,
    agency: pkg.agency, // need to include agency in the query
    amount,
    existingPayment,
  };
}
```

2.4 Case 2: User Plan + Agency Offer (Negotiation Flow)

This flow already mostly works but needs the fee/split layer added:

```
FLOW: User Plan → Agency Offer → Negotiation → Payment

[1] User creates plan → publishes → OPEN
[2] Other users join (group forms)
[3] Agencies discover plan and submit offers (inline in group chat)
[4] Creator negotiates: counter-offers (max 3 rounds)
[5] Creator accepts offer → plan moves to CONFIRMING

[6] All members get payment prompt (48h window)
    Same fee structure as package flow:
      Trip cost + ₹299 + GST = Total
      Split via Razorpay Route

[7] CRITICAL: What if some pay and some don't?

    Scenario A: 6 of 8 pay within 48h
    → IF 6 >= groupSizeMin: plan CONFIRMED with 6 members
       Non-payers auto-removed, waitlist notified
    → IF 6 < groupSizeMin: plan reverts to OPEN
       All 6 payers auto-refunded (full including ₹299)
       Agency notified: 'Group didn’t reach minimum. Plan reopened.'

    Scenario B: All 8 pay
    → CONFIRMED. Group locked. Trip proceeds.

    Scenario C: 0 pay within 48h
    → Plan reverts to OPEN. Offer remains valid if not expired.

[8] After CONFIRMED:
    Agency payout schedule: Trust Mode or Pro Mode
    Escrow releases per schedule
    Reviews unlocked after trip
```

2.5 Agency Wallet & Ledger System

This is the single biggest missing piece for agency retention. Without financial visibility, agencies will demand direct payments.

New Prisma Models Required

```
model AgencyWallet {
  id              String   @id @default(uuid())
  agencyId        String   @unique
  agency          Agency   @relation(fields: [agencyId], references: [id])
  pendingBalance  Int      @default(0) // paise, awaiting release
  availableBalance Int     @default(0) // paise, released + ready
  totalEarned     Int      @default(0) // paise, lifetime
  totalCommission Int      @default(0) // paise, lifetime
  securityDeposit Int      @default(0) // paise, Pro mode deposit
  payoutMode      String   @default("trust") // trust | pro
  updatedAt       DateTime @updatedAt

  transactions    AgencyTransaction[]
  @@map("agency_wallets")
}

model AgencyTransaction {
  id          String   @id @default(uuid())
  walletId    String
  wallet      AgencyWallet @relation(fields: [walletId], references: [id])
  type        String   // booking_received | tranche_released | commission_deducted
                       // | refund_deducted | payout_settled | deposit_held | deposit_returned
  amount      Int      // paise (positive = credit, negative = debit)
  description String
  groupId     String?  // linked to which trip
  paymentId   String?  // linked to which payment
  razorpayTransferId String?
  createdAt   DateTime @default(now())

  @@index([walletId, createdAt(sort: Desc)])
  @@map("agency_transactions")
}
```

The agency dashboard shows a clean wallet view:

- Pending: ₹1,84,000 (3 trips in progress, escrow held)
- Available: ₹46,000 (1 tranche released, ready for payout)
- Settled: ₹2,30,000 (lifetime earnings after commission)
- Commission paid: ₹20,000 (lifetime platform fees)
- Transaction history: every credit and debit with trip reference, date, and Razorpay transfer ID.

2.6 Structured Cancellation Policy

Your Offer model stores cancellationPolicy as free text. This is unenforceable. Replace with structured data:

```
// In the Offer model, replace cancellationPolicy String? with:

cancellationRules  Json?
// Structure:
// {
//   rules: [
//     { daysBeforeTrip: 30, refundPercent: 100 },
```

```
//     { daysBeforeTrip: 14, refundPercent: 75 },
//     { daysBeforeTrip: 7,  refundPercent: 50 },
//     { daysBeforeTrip: 3,  refundPercent: 25 },
//     { daysBeforeTrip: 0,  refundPercent: 0 },
//   ],
//   convenienceFeeRefundable: false,
//   agencyCancelFullRefund: true,
// }
```

When a user requests cancellation, the system auto-calculates the refund amount by checking how many days are left before the trip start date. No manual intervention needed.

3. Agency Team System

3.1 Why This Matters

Your current Agency model has a single ownerId. This means the agency owner must personally handle every offer, every chat message, every booking, and every payout. For an agency with 5–10 employees, this is unsustainable. They'll delegate to WhatsApp groups internally, which means platform communication leaks.

3.2 New Prisma Model

```
model AgencyMember {
  id        String   @id @default(uuid())
  agencyId  String
  userId    String
  agency    Agency   @relation(fields: [agencyId], references: [id])
  user      User     @relation(fields: [userId], references: [id])
  role      String   // admin | manager | agent | finance
  isActive  Boolean  @default(true)
  invitedBy String?  // userId of who invited them
  joinedAt  DateTime @default(now())

  @@unique([agencyId, userId])
  @@map("agency_members")
}
```

3.3 Role Permissions

Permission	Admin (Owner)	Manager	Agent	Finance
Edit agency profile + storefront	✓	✓	✗	✗
Create/edit packages	✓	✓	✓	✗
Submit offers on user plans	✓	✓	✓	✗
Accept/reject counter-offers	✓	✓	✗	✗
Chat with users (group + DM)	✓	✓	✓	✗
View bookings	✓	✓	✓	✓
View wallet + transactions	✓	✓	✗	✓
Request payouts	✓	✗	✗	✓
Invite/remove team members	✓	✓	✗	✗
View analytics	✓	✓	✓	✓
Change payout mode (Trust/Pro)	✓	✗	✗	✗
Delete agency	✓	✗	✗	✗

MVP Decision: Defer to Phase 3

- For MVP (Phase 1–2), keep single-owner agencies. The ownerId model works fine for small agencies.
- Build the AgencyMember model but don't build the invitation/role UI until Phase 3.
- This is a 'nice to have' for launch, not a 'must have'. Focus on payment and chat fixes first.

4. User → Agency Flow After Offer Acceptance (Step by Step)

```
COMPLETE POST-ACCEPTANCE FLOW

STATE: OPEN (offer just accepted by creator)
│
├→ [Backend] Plan status: OPEN → CONFIRMING
├→ [Backend] Agency rep added to group chat as special member
├→ [Chat] System message: '✓ Offer accepted! [Agency] joined the group.'
├→ [Chat] Payment prompt card appears for all members
├→ [Backend] 48h timer scheduled (BullMQ delayed job)
├→ [Notification] Push + email to all members: 'Trip ready! Pay within 48h'
├→ [Notification] Push to agency: 'Your offer was accepted for [Plan Title]'
│
STATE: CONFIRMING (48h payment window)
│
├→ Each user pays individually via Razorpay Checkout
├→ [Chat] System message per payment: 'Priya paid (3/8 confirmed)'
├→ [Chat] Progress bar updates in real-time
├→ [Backend] After each payment: check if all APPROVED members have paid
│
├→ IF all paid:
│    ├→ Plan status: CONFIRMING → CONFIRMED
│    ├→ Group locked (isLocked = true)
│    ├→ [Chat] System: '✓ All payments received! Trip confirmed!'
│    ├→ [Chat] Trip Contact Card appears (masked agency phone)
│    ├→ [Chat] Agency can now share documents (itinerary, bookings)
│    ├→ [Backend] Schedule escrow tranche 1 for trip start date
│    ├→ [Backend] Schedule escrow tranche 2 for trip end + 48h
│    └→ [Notification] All members: 'Trip confirmed! Check chat for details.'
│
├→ IF 48h expires with some unpaid:
│    ├→ IF paid_count >= groupSizeMin:
│    │    Remove unpaid members, CONFIRMED with smaller group
│    └→ IF paid_count < groupSizeMin:
│         Refund all payers, revert to OPEN, notify everyone
│
STATE: CONFIRMED (trip coordination phase)
│
├→ Agency shares: itinerary PDF, hotel booking PDFs, meeting point
├→ Members coordinate: questions, requests, polls
├→ [Chat] Pinned section: Trip Info (dates, contact, docs)
│
STATE: TRIP IN PROGRESS
│
├→ Agency marks 'Trip Started' in dashboard
├→ At least 1 member confirms via app
├→ [Backend] Tranche 1 released (50% to agency)
├→ [Wallet] Transaction: 'Tranche 1 released: ₹46,000'
│
STATE: COMPLETED
│
```

```
├→ 48h after trip end: Tranche 2 auto-released (unless disputed)
├→ [Chat] Review prompts appear for all members
├→ Reviews: rate agency (overall + safety + value) + rate co-travelers
├→ [Wallet] Transaction: 'Tranche 2 released: ₹46,000'
└→ Chat remains accessible for 30 days, then read-only
```

5. Edge Cases: Every Scenario That Can Go Wrong

Edge Case	Current Handling	Fix
User pays but others don't (48h expires)	Not handled. No timeout job exists.	BullMQ delayed job at T+48h. If paid_count >= min: confirm with smaller group. If < min: auto-refund all, revert to OPEN.
Group never fills (package never reaches min)	Package stays OPEN forever.	Auto-expire packages 7 days before startDate if min not met. Notify all interested members. Creator can re-publish.
Agency cancels after payment	Not handled.	Full refund to all users (including ₹299). Agency penalized: rating deduction (-0.5), warning flag. 2 cancellations = suspension.
User cancels after CONFIRMED	Not handled.	Apply structured cancellation rules. Refund per daysBeforeTrip policy. Notify group: 'Member left. X/Y remaining.' If below min: agency decides whether to proceed or cancel.
Payment success but Razorpay webhook delayed	razorpayWebhookHandler exists but no reconciliation.	Add BullMQ job every 6 hours: fetch all PENDING payments, check Razorpay order status via API, update any that are actually captured.
Duplicate payment (user taps Pay twice)	Partially handled (checks existingPayment).	Enforce idempotency: if payment exists with CAPTURED status, return success immediately. If PENDING, check Razorpay status first.
Agency tries to move user off-platform	No detection.	Contact masking (Section 1.2). System warnings. Repeated violations (3+ flagged messages) trigger admin review and temporary chat restriction.
User asks agency for direct payment	No detection.	Same masking. Plus: if user sends UPI/bank info, auto-inject warning to BOTH parties. Log for admin.
Refund dispute (user claims bad trip)	No dispute model.	Add Dispute model: user files dispute before tranche 2 release. Hold tranche 2. Both parties submit evidence. Admin resolves within 7 days. If unresolved: split 50/50.
Agency becomes inactive mid-trip	No monitoring.	If agency rep doesn't open the chat for 72h after CONFIRMED, escalate to admin. Auto-notification to agency owner. If no response in 24h more: platform contacts agency directly.
Razorpay Route payout fails	Not handled.	Retry logic with exponential backoff (3 attempts over 24h). If all fail: admin alert + manual payout. Log in AgencyTransaction as 'payout_failed'.
User joins two conflicting trips (same dates)	Allowed with warning.	At join time: check existing CONFIRMED trips for date overlap. Show warning: 'You have a confirmed trip to Manali on these dates.' Don't block, just warn.

6. Phase-Wise Implementation Plan

Phase 1: MVP Fix (Weeks 1–3)

Minimum changes to make the current system actually transactable.

- Fix getGroupPaymentContext() to support package flow (not just plans with offers).
- Add convenience fee (₹299 + GST) as separate line item in payment creation.
- Add contact masking to chat service (sanitizeChatContent function).
- Add 48h payment timeout BullMQ job.
- Add payment reconciliation job (every 6 hours, check Razorpay for stuck payments).
- Add structured cancellationRules to Offer model (replace free-text policy).
- Add system warning messages when contact info is detected in chat.

Phase 2: Stability (Weeks 4–7)

Make payments reliable and agency experience professional.

- Integrate Razorpay Route for actual split payments (platform fee + commission + agency payout).
- Build AgencyWallet + AgencyTransaction models and the wallet dashboard.
- Build Invoice model: auto-generate PDF invoices for every confirmed booking.
- Build Dispute model and the dispute resolution flow.
- Add virtual number masking for post-payment agency contact sharing (Exotel/Knowlarity).
- Implement Trust Mode payout schedule (50/50 tranche release via Razorpay Transfer API).

Phase 3: Scale (Weeks 8–12)

Agency team features and advanced payment options.

- Build AgencyMember model with role-based permissions.
- Agency team invitation flow (email invite, role assignment).
- Pro Mode payout system (30/30/32 split with advance release).
- Agency analytics dashboard: revenue, conversion rates, demand heatmap, bid win rate.
- Booking calendar with trip status, member list, payout schedule.

Phase 4: Advanced (Weeks 13–16)

- Agency trust score algorithm (based on rating, completion rate, response time, dispute rate).
- Automated trust-to-pro graduation.
- Travel insurance integration (Digit/Acko) at checkout.
- Agency CRM: customer database, re-engagement campaigns, seasonal promotions.
- Fraud detection engine: behavioral analysis for suspicious patterns.

7. Platform Protection Strategy

7.1 Revenue Capture Safeguards

Layer	Mechanism	When Active
Fee at source	The ₹299 + GST is collected at checkout as a separate Razorpay Route split. It goes directly to TravellersIn's settlement account. Not refundable on user cancellation (except agency-cancel).	At every payment
Commission at payout	8% deducted from each escrow tranche release. Agency sees net amount in wallet. Deduction is automated via Razorpay Transfer API.	At every tranche release
Escrow as leverage	If communication moves off-platform, the user loses escrow protection. This is the #1 retention mechanism. Show it prominently.	Throughout the trip lifecycle
Review lock-in	Only platform-transacted trips can be reviewed. Reviews are visible on profiles and affect search ranking. This is social capital that can't be replicated off-platform.	After trip completion
Document lock-in	Itineraries, hotel bookings, permits are shared as platform documents. If a user goes off-platform, they lose access to these files.	After CONFIRMED status

7.2 Anti-Bypass UX Nudges

- Chat warning banner: always-visible at the top of every chat: 'Your booking is protected by TravellersIn escrow. External payments are not covered.'
- Payment page safety notice: 'If anyone asks you to pay outside the platform, please report them. External payments are not refundable.'
- Post-payment confirmation: 'Your ₹12,853 is safely in escrow. The agency receives payment only after the trip starts.'
- First-message nudge (in DMs): when a user or agency sends their first DM, a system message appears: 'All transactions should happen through TravellersIn for your protection.'

7.3 Agency Retention Strategy

- Commission at 8% is competitive: MakeMyTrip charges 15–25%. Goibibo charges 12–20%. You're the cheapest legitimate platform.
- Qualified leads: agencies get structured demand with dates, budgets, headcount, and preferences. No other platform provides this.
- Wallet visibility: clear financial dashboard beats the chaos of tracking direct payments via bank statements.
- Analytics: bid win rate, demand heatmap, response time metrics. Small agencies have never had access to this data.
- Pro Mode incentive: the promise of faster payouts motivates agencies to maintain high quality.

Final Reality Check

- The #1 risk is still disintermediation. No amount of technical safeguards will prevent a determined agency from sharing their phone number verbally during the trip. The goal is not to make it impossible — it's to make staying on-platform MORE CONVENIENT than leaving.
- The escrow protection story is your strongest weapon. 'Your money is safe because we hold it until the trip happens' — this one sentence is worth more than any technical feature.

- Start with Phase 1 fixes (3 weeks of work) and the platform becomes actually transactable. Without the package payment fix and the convenience fee split, you literally cannot collect revenue.
- The agency wallet (Phase 2) is what turns agencies from reluctant participants into advocates. When an agency can see their money and know exactly when it arrives, they stop fighting the platform and start promoting it.